



# Case Study – A Physics-Informed Hydraulic Surrogate Model

MSc. Inaam Ashraf, MSc. Luca Hermes, Dr. André Artelt

[{aartelt,mashraf,lhermes}@techfak.uni-bielefeld.de](mailto:{aartelt,mashraf,lhermes}@techfak.uni-bielefeld.de)

*WDSA CCWI 2026*

Paphos, 18/05/2026



Water-Futures has received funding from the European Research Council (ERC) under the European Union's Horizon 2020 research and innovation programme (Grant agreement No. 951424)

This research was supported by the Ministry of Culture and Science NRW (Germany) as part of the Lamarr Fellow Network. This publication reflects the views of the authors only.

# Outline of Today

| Time  | Topics                                                             |
|-------|--------------------------------------------------------------------|
| 14:00 | Introduction & Welcome                                             |
|       | Surrogate models vs. digital twins                                 |
|       | Types & Implementations of surrogates                              |
| 14:15 | Recap of Machine Learning (ML) and Deep Learning                   |
|       | Basic concepts, evaluation, hyper-parameter tuning, etc.           |
|       | Surrogate models: Challenges for ML                                |
|       | Data-driven mitigation strategies (i.e., scenario generation)      |
| 14:45 | Physics-informed/inspired Methods -- Part I                        |
|       | Intro to Physics-informed Neural Networks (PINNs)                  |
|       | <b>Hands-On:</b> How to build and train a PINN in Python           |
| 15:30 | Coffee Break                                                       |
| 16:00 | Physics-informed/inspired Methods -- Part II                       |
|       | Operators vs. PINNs                                                |
| 16:25 | Case study: SOTA Hydraulic Surrogate Model                         |
|       | Hydraulic surrogates and their applications                        |
|       | Analyzing the problem and identifying relevant quantities and laws |
|       | A SOTA Hydraulic Surrogate                                         |
|       | <b>Hands-On:</b> Usage and Evaluation of the Surrogate in Python   |
| 17:15 | Closing, Summary, and Q&A                                          |
| 17:30 | END                                                                |

We are here!





- Background
- Physics-Informed GNN-based Surrogate Model
- Challenges in Physics-Informed Learning
- Towards Digital Twins
- Hands on Session



# Background

# Hydraulic Principles<sup>1</sup>

- Flow direction:

$$q_{e_{uv}}^* = -q_{e_{vu}}^*$$

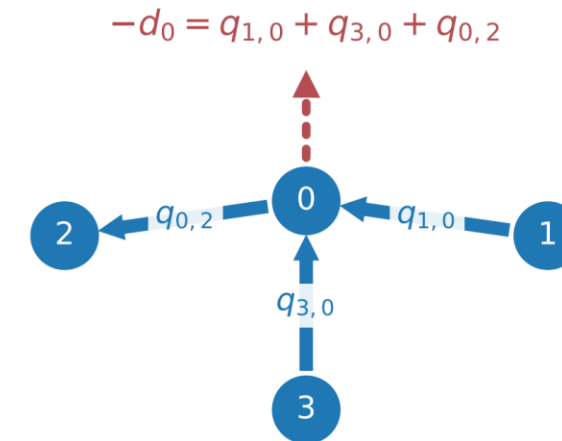
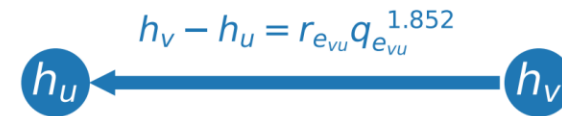
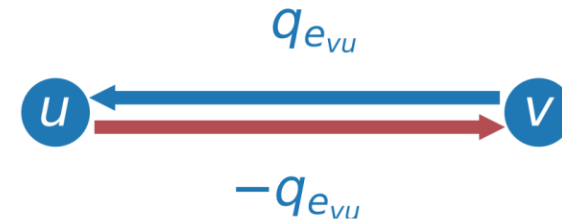
- Law of conservation of energy – Head Loss:

$$h_v^* - h_u^* = r_{e_{vu}} \operatorname{sgn}(q_{e_{vu}}^*) |q_{e_{vu}}^*|^{1.852}$$

$$h_v^* - h_u^* = \omega^2 (\iota - \kappa (-q_{e_{vu}}^* / \omega)^\nu)$$

- Law of conservation of mass:

$$\sum_{u \in \mathcal{N}(v)} q_{e_{vu}}^* = -d_v^*.$$



# Problem Statement – Hydraulic State Simulation

**Definition 1.** *Given a WDS graph  $\mathcal{G} := (V, E)$ , the task of hydraulic steady state simulation involves estimating the pressure head at every node  $(h_v)_{v \in V}$  and water flow through every link  $(q_e)_{e \in E}$  given demands at every consumer node  $(d_v^*)_{v \in V_c}$ , pressure heads at the reservoirs  $(h_v^*)_{v \in V_r}$ , and edge attributes (e.g.,  $(r_e)_{e \in E_{pipe}}$ ,  $(\omega_e, \iota_e, \kappa_e, \nu_e)_{e \in E_{pump}}$ ).*

- The problem is formulated as a system of non-linear equations:

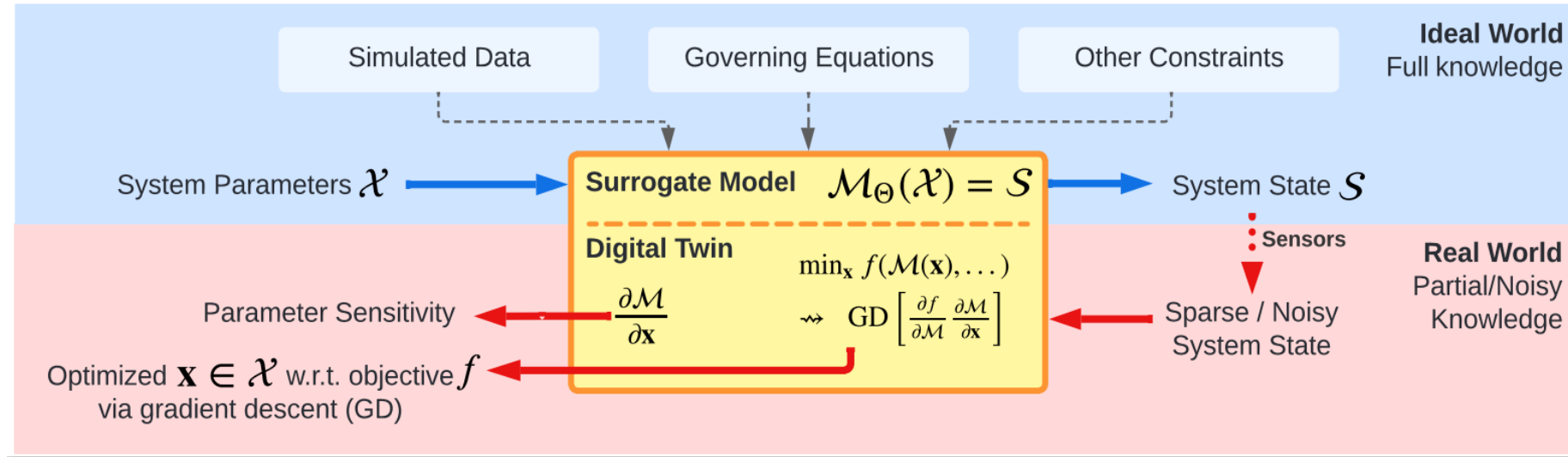
$$\mathbf{d}^* := -\mathbf{A}\mathbf{q}^*, \quad \Delta \mathbf{h}^* := \mathbf{R}^*(\mathbf{q}^*)^x + \mathbf{S}\omega^{*2}\boldsymbol{\iota}^* - \mathbf{S}\boldsymbol{\kappa}^*(\mathbf{q}^*/\omega^*)^{\boldsymbol{\nu}^*}$$

- Although the hydraulic principles are local, the problem has global complexity.

# Hydraulic Simulation

- EPANET is the current SOTA hydraulic simulator<sup>1</sup>.
- It solves the aforementioned system of non-linear equations iteratively.
- The computational cost can become high for longer times / larger WDSs.
- It is not differentiable.

# Surrogates and Digital Twins

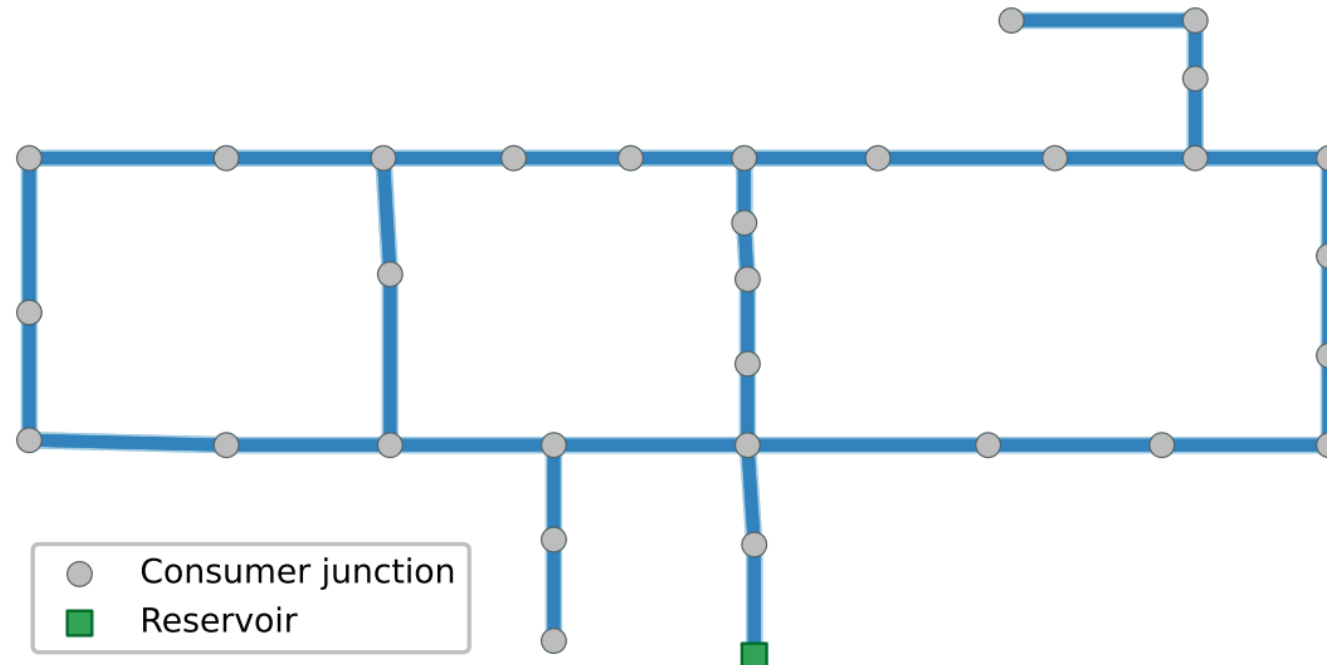


- A Deep Learning (DL) Surrogate Model is end-to-end differentiable<sup>1,2</sup>.
- A surrogate model can serve as a Digital Twin given some assumptions<sup>3</sup>.



# WDS as a Graph

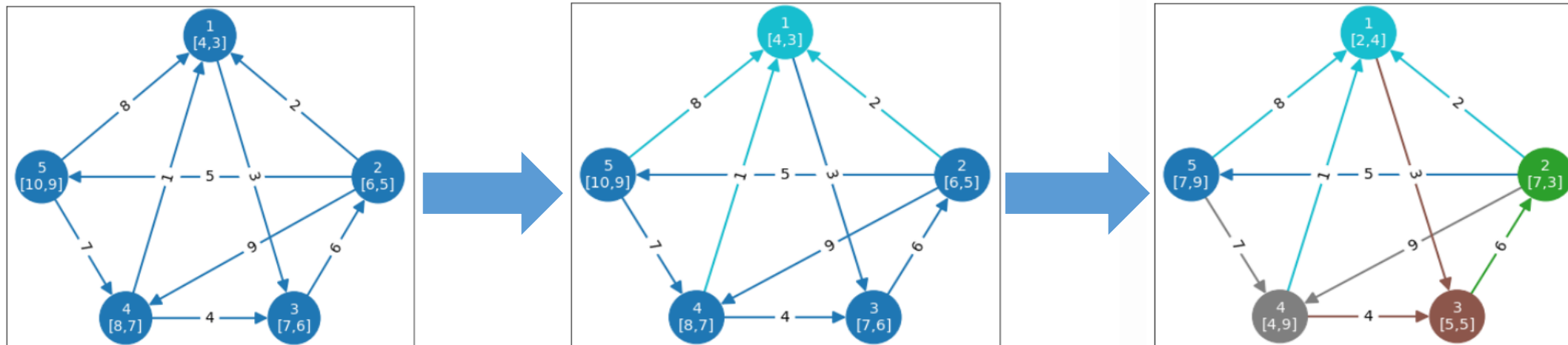
**Water Distribution System as a Graph** : A Water Distribution System (WDS) is a heterogeneous graph  $\mathcal{G} := (V, E)$  with certain node types such as consumer junctions  $V_c$  and water sources/reservoirs (lakes, rivers, etc.)  $V_r$ , where  $V = V_c \cup V_r$ , and certain edge types such as pipes  $E_{pipe}$ , pumps  $E_{pump}$ , and valves  $E_w$ , where  $E = E_{pipe} \cup E_{pump} \cup E_w = \{e_{vu} \mid \forall v \in V; u \in \mathcal{N}(v)\}$  with cardinality  $|V| = N_n$  and  $|E| = N_e$  respectively.



# Modeling Graphs using ML

- Intuitive choice – Graph Neural Networks
- Spectral GNNs<sup>1</sup>
  - Based on graph spectral theory (Graph Laplacian, Eigen Vectors).
  - Computationally expensive.
- Spatial GNNs<sup>2</sup>
  - Based on message passing.
  - Local operators.

# Message Passing Graph Neural Networks



- Message Construction:
- Message Aggregation:
- Feature Update:

$$\mathbf{m}_{e_{vu}} = \mathbf{x}_u \parallel f_{e_{vu}}, u \in \mathcal{N}(v)$$

$$\mathbf{m}_v = \sum_{u \in \mathcal{N}(v)} \mathbf{m}_{e_{vu}}$$

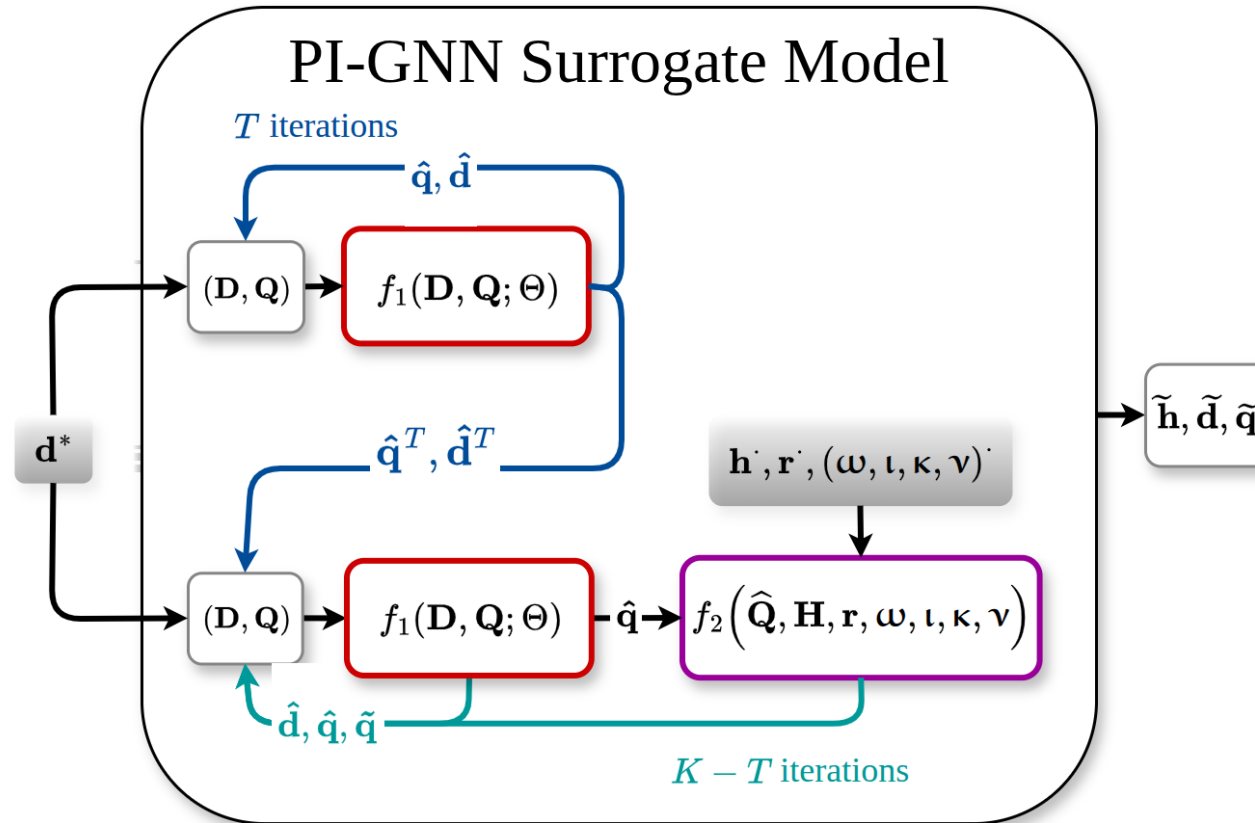
$$\hat{\mathbf{x}}_v = \phi(\mathbf{m}_v) = MLP(\mathbf{m}_v)$$



# Physics-Informed GNN-based Surrogate Model

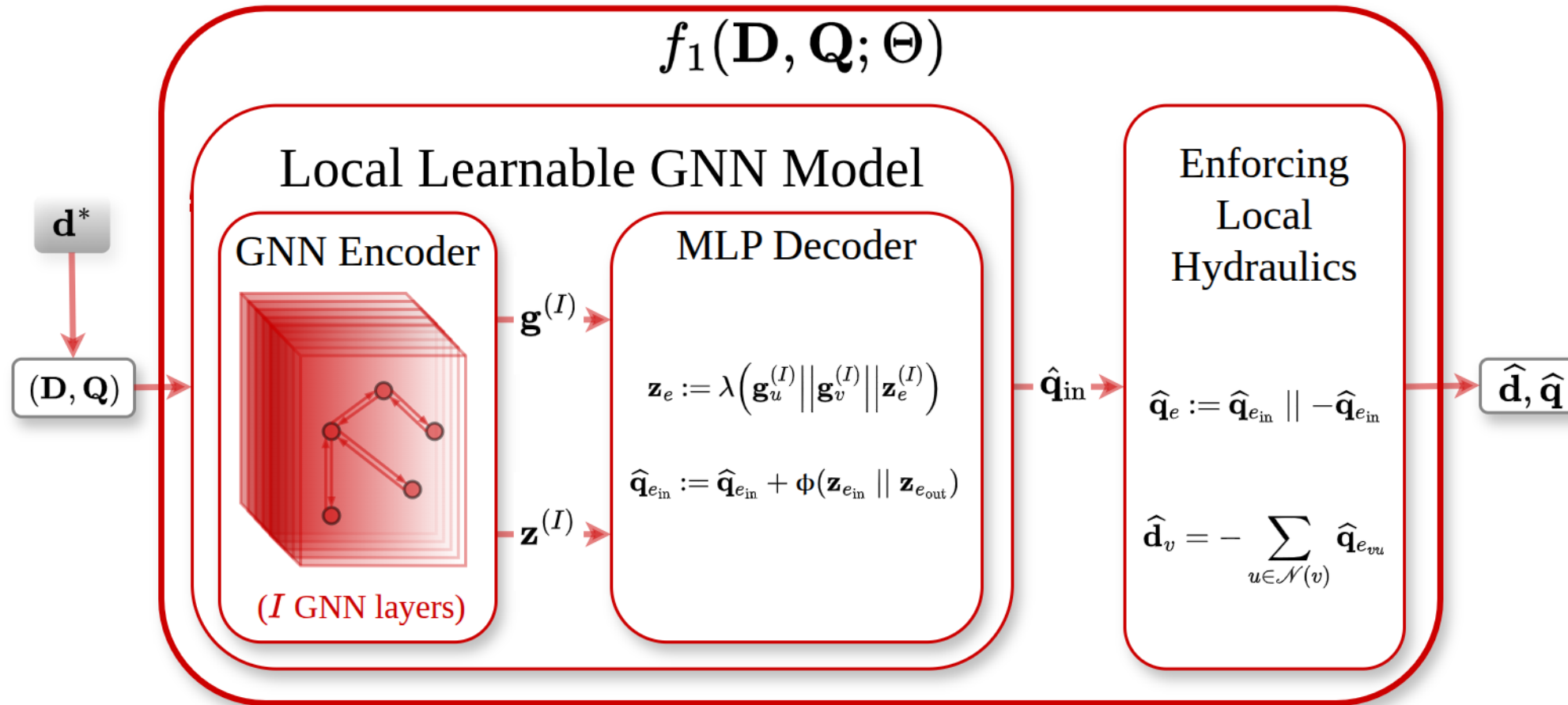
# Physics-Informed GNN-based Surrogate Model<sup>1,2</sup>

$$\mathbf{d}^* := -\mathbf{A}\mathbf{q}^*, \quad \Delta \mathbf{h}^* := \mathbf{R}^*(\mathbf{q}^*)^x + \mathbf{S}\omega^{*2}\iota^* - \mathbf{S}\kappa^*(\mathbf{q}^*/\omega^*)^{\nu^*}$$



1. <https://doi.org/10.1609/aaai.v38i20.30192>
2. [10.1109/IJCNN64981.2025.11229349](https://doi.org/10.1109/IJCNN64981.2025.11229349)

# Local Learnable GNN Module



# Local Learnable GNN Module

- Edge-based GNN Encoder

Message Construction:

$$\mathbf{m}_e^{(i)} := \text{ReLU}(\mathbf{z}_e^{(i)} + \gamma^{(i)}(\mathbf{g}_u^{(i)} \parallel \mathbf{g}_v^{(i)} \parallel \mathbf{z}_e^{(i)})) \text{ with } u \in \mathcal{N}(v).$$

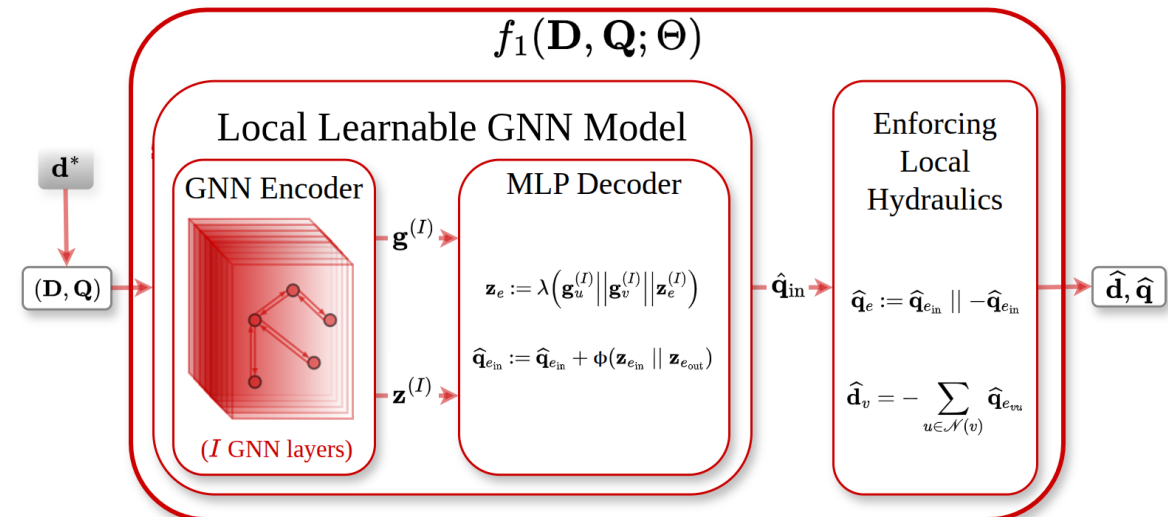
Message Aggregation:

$$\mathbf{m}_v^{(i)} := \max_{u \in \mathcal{N}(v)} \mathbf{m}_{e_{vu}}^{(i)}.$$

Feature update:

$$\mathbf{g}_v^{(i+1)} := \mathbf{m}_v^{(i)}$$

$$\mathbf{z}_e^{(i+1)} := \mathbf{m}_e^{(i)}.$$



# Local Learnable GNN Module

- MLP Decoder

$$\bar{\mathbf{z}}_e := \lambda(\mathbf{g}_u^{(I)} \parallel \mathbf{g}_v^{(I)} \parallel \mathbf{z}_e^{(I)}) \text{ with } u \in \mathcal{N}(v), \forall e \in E$$

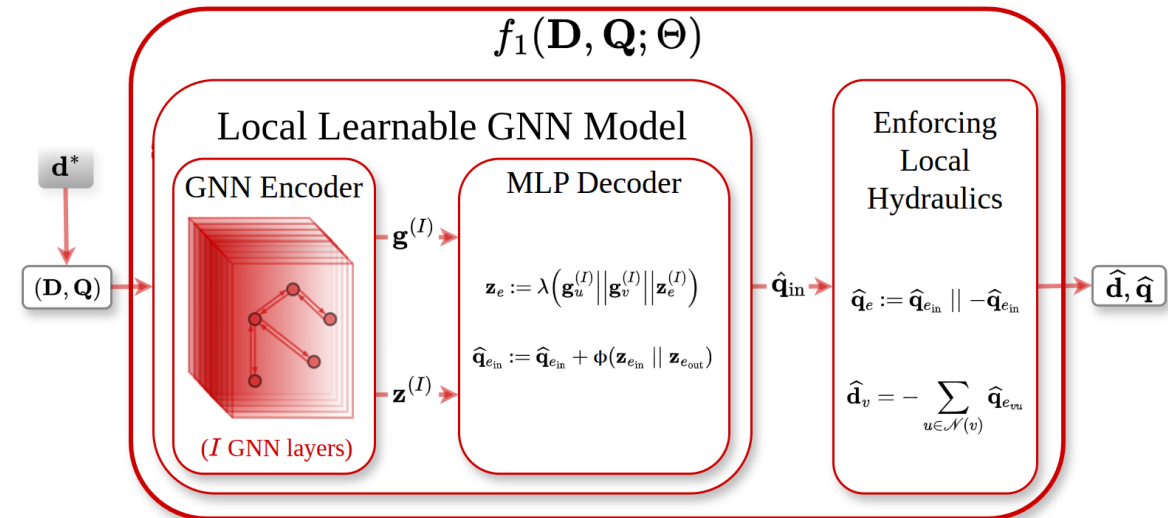
$$\bar{q}_{e_{in}} := \bar{q}_{e_{in}} + \phi(\bar{\mathbf{z}}_{e_{in}} \parallel \bar{\mathbf{z}}_{e_{out}}), \quad \forall e \in E.$$

- Enforcing Flow Direction Property

$$\hat{q}_e := \bar{q}_{e_{in}} \parallel (-\bar{q}_{e_{in}}), \quad \forall e \in E.$$

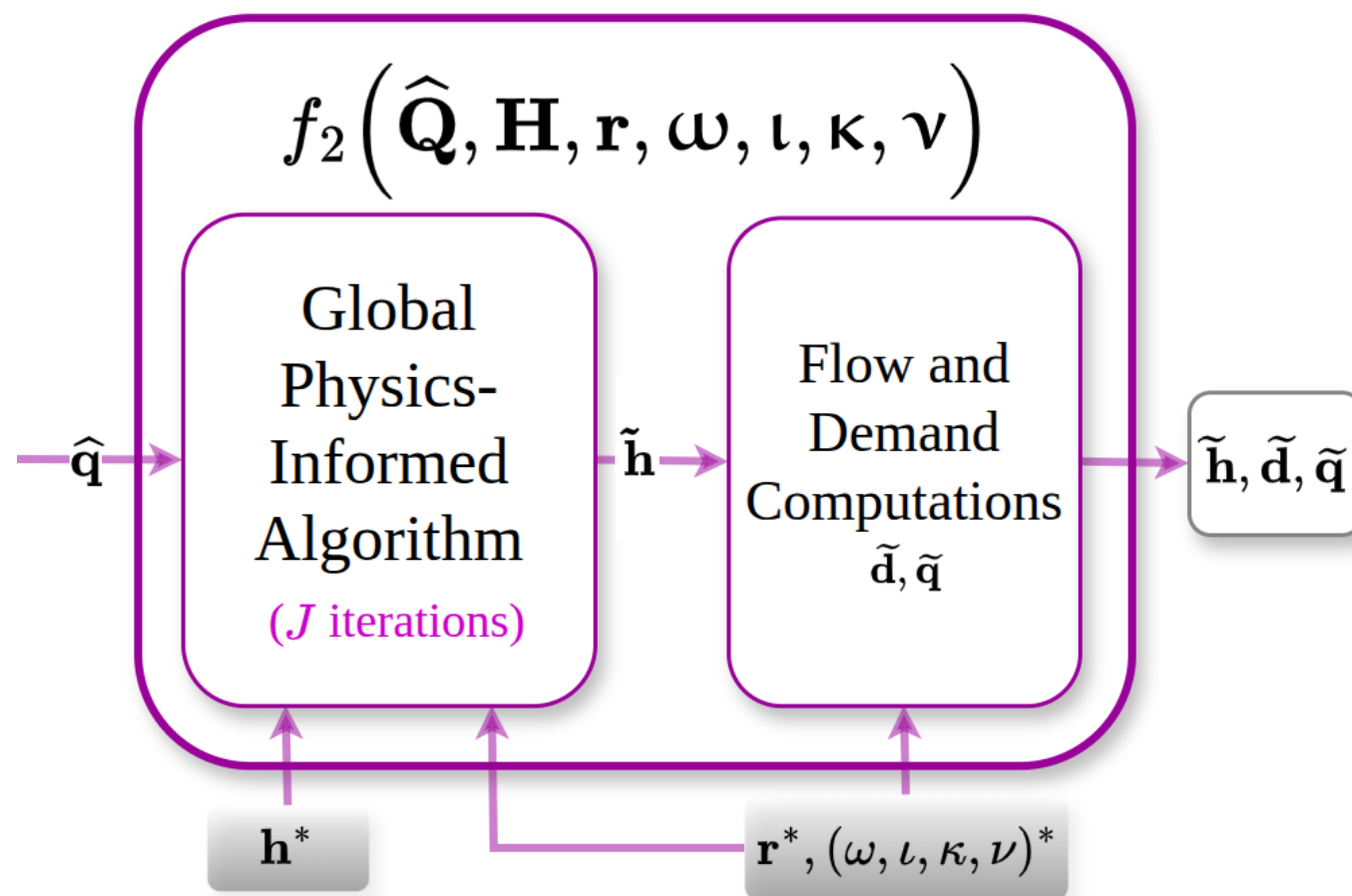
- Computing demands

$$\hat{d}_v := - \sum_{u \in \mathcal{N}(v)} \hat{q}_{e_{vu}}.$$





# Global Physics-Informed Module



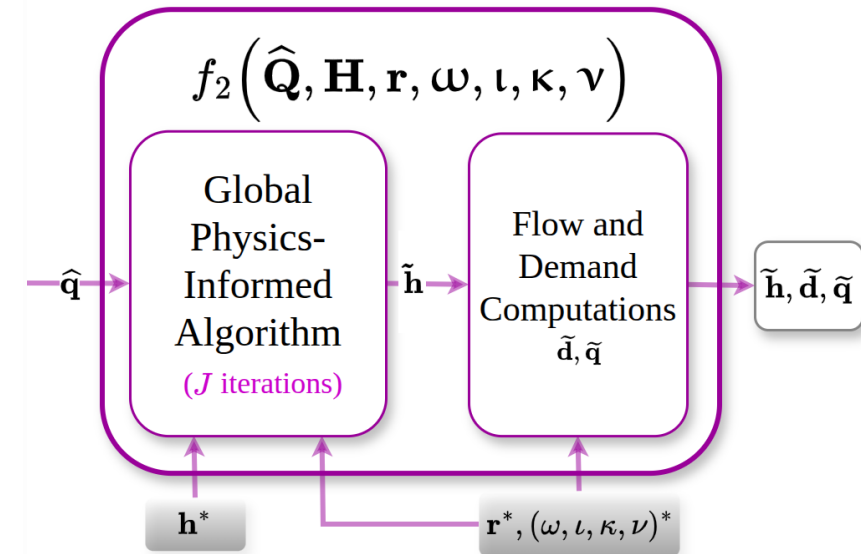
# Global Physics-Informed Module

- Global Physics-Informed Algorithm

Message Construction:  $m_e^{(j)} := \tilde{h}_u^{(j)} - f_{ql}(q_e, l_e), \forall e \in E$

Message Aggregation:  $m_v^{(j)} := \max_{u \in \mathcal{N}(v)} m_{e_{vu}}^{(j)}, \forall v \in V \setminus V_r$

Feature update:  $\tilde{h}_v^{(j+1)} := \max\{\tilde{h}_v^{(j)}, m_v^{(j)}\}, \forall v \in V \setminus V_r$



# Global Physics-Informed Module

- Global Physics-Informed Algorithm

Message Construction:  $m_e^{(j)} := \tilde{h}_u^{(j)} - f_{ql}(q_e, l_e), \forall e \in E$

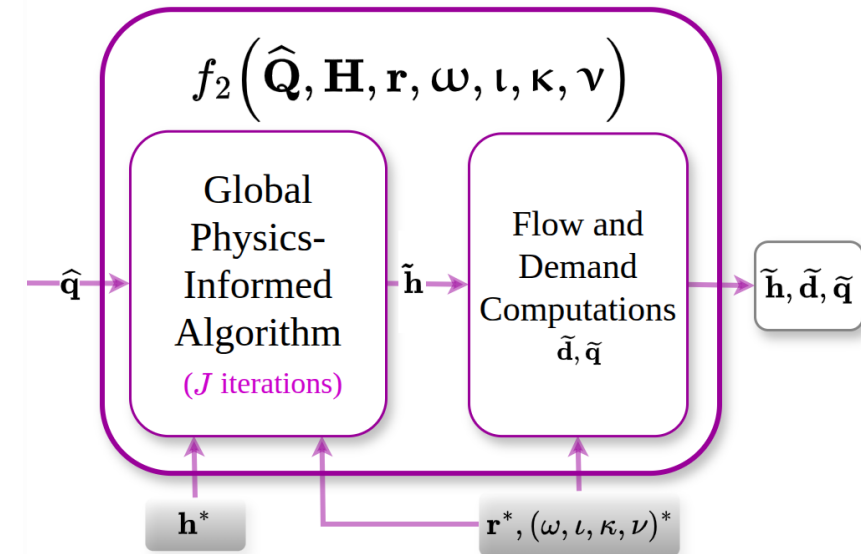
Message Aggregation:  $m_v^{(j)} := \max_{u \in \mathcal{N}(v)} m_{e_{vu}}^{(j)}, \forall v \in V \setminus V_r$

Feature update:  $\tilde{h}_v^{(j+1)} := \max\{\tilde{h}_v^{(j)}, m_v^{(j)}\}, \forall v \in V \setminus V_r$

- Flow and demand computation

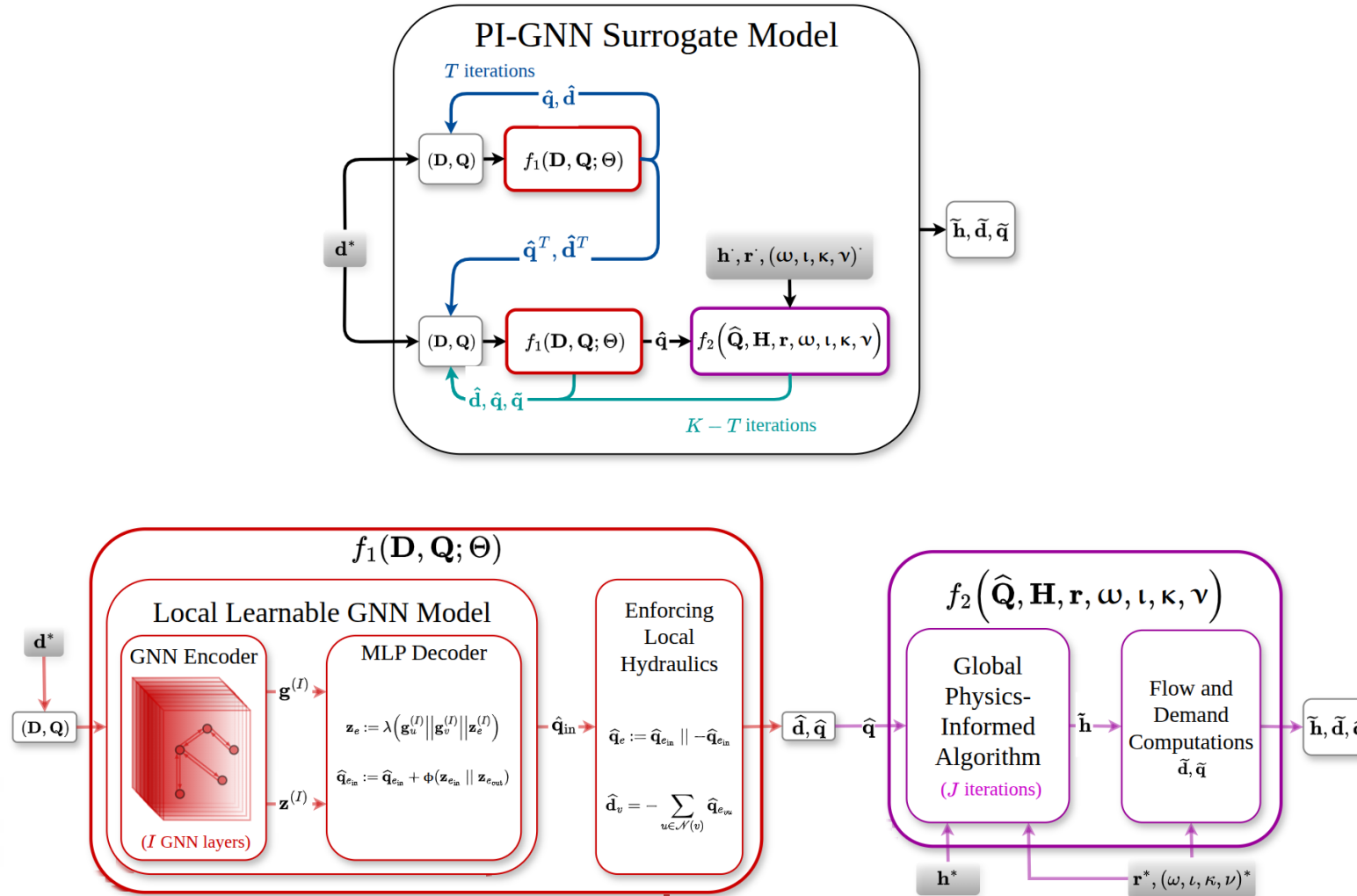
$$\tilde{q}_{e_{vu}} := f_{hl}(\tilde{h}_v, \tilde{h}_u, l_{e_{vu}}), \forall e_{vu} \in E$$

$$\tilde{d}_v := -\sum_{u \in \mathcal{N}(v)} \tilde{q}_{e_{vu}}, \forall v \in V$$



# Optimization Objective

$$\mathcal{L} = \mathcal{L}(\mathbf{d}^*, \hat{\mathbf{d}}^{(K)}) + \rho \mathcal{L}(\mathbf{d}^*, \tilde{\mathbf{d}}^{(K)}) + \delta \mathcal{L}(\hat{\mathbf{q}}^{(K)}, \tilde{\mathbf{q}}^{(K)})$$



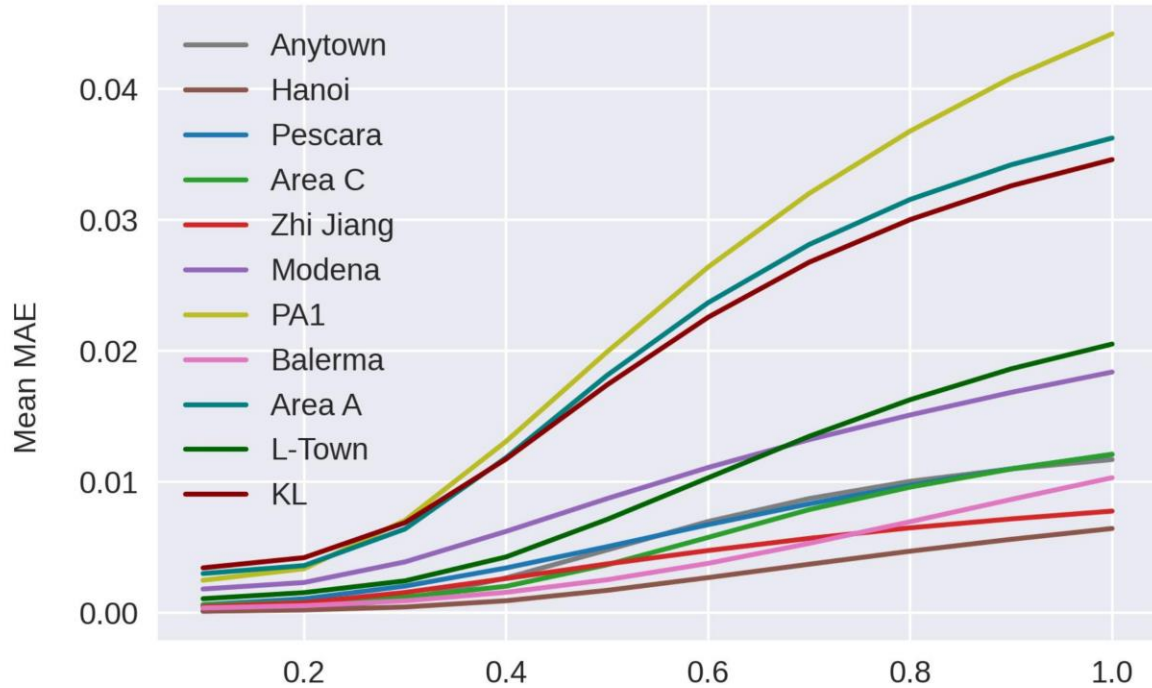
# Results – Errors

| MAE<br>( $\times 10^{-2}$ ) | Anytown           | Hanoi             | Pescara           | L-Town<br>Area C    | Zhi Jiang         |
|-----------------------------|-------------------|-------------------|-------------------|---------------------|-------------------|
| <i>Demands</i>              | $0.019 \pm 0.029$ | $0.012 \pm 0.023$ | $0.068 \pm 0.129$ | $0.126 \pm 0.025$   | $0.090 \pm 0.160$ |
| <i>Flows</i>                | $0.005 \pm 0.009$ | $0.001 \pm 0.002$ | $0.011 \pm 0.017$ | $0.005 \pm 0.007$   | $0.003 \pm 0.005$ |
| <i>Heads</i>                | $0.001 \pm 0.001$ | $0.013 \pm 0.005$ | $0.052 \pm 0.068$ | $0.028 \pm 0.024$   | $0.028 \pm 0.019$ |
| MAE<br>( $\times 10^{-2}$ ) | Modena            | PA1               | Balerna           | L-Town All<br>Areas | KL                |
| <i>Demands</i>              | $0.361 \pm 1.874$ | $0.264 \pm 0.773$ | $0.092 \pm 0.442$ | $0.278 \pm 0.668$   | $0.566 \pm 2.501$ |
| <i>Flows</i>                | $0.019 \pm 0.085$ | $0.043 \pm 0.098$ | $0.001 \pm 0.002$ | $0.028 \pm 0.050$   | $0.033 \pm 0.193$ |
| <i>Heads</i>                | $0.124 \pm 0.121$ | $0.344 \pm 0.273$ | $0.007 \pm 0.006$ | $0.024 \pm 0.023$   | $0.534 \pm 0.371$ |

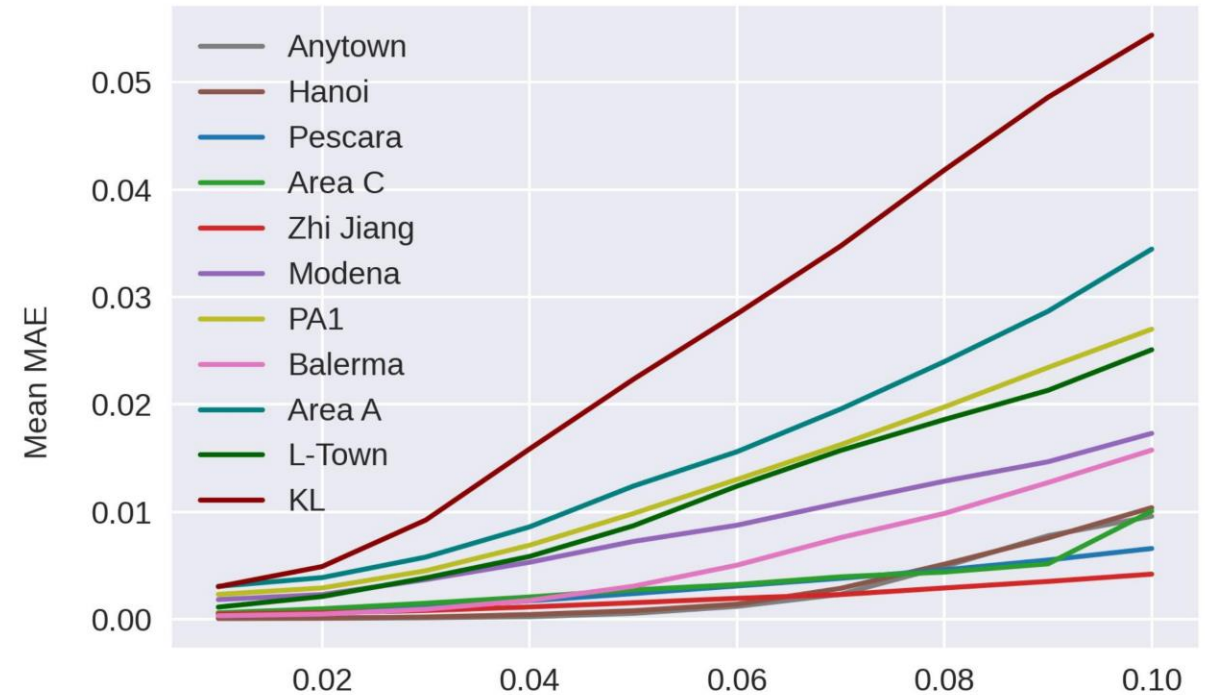
# Results – Computation Times

| Standard Deviation<br>( $\times d_{\text{base}}$ ) | KL  | PA1   | Balerna | Modena | Pescara |
|----------------------------------------------------|-----|-------|---------|--------|---------|
| 0.1                                                | 190 | 557   | 1,221   | 1,640  | 2,651   |
| 0.2                                                | 245 | 1,086 | 2,595   | 3,280  | 6,647   |
| 0.3                                                | 287 | 1,713 | 3,942   | 4,834  | 9,272   |
| 0.4                                                | 339 | 2,218 | 5,465   | 6,299  | 11,328  |
| 0.5                                                | 375 | 2,713 | 6,881   | 7,194  | 13,708  |
| 0.6                                                | 407 | 3,104 | 8,183   | 8,471  | 13,742  |
| 0.7                                                | 437 | 3,432 | 9,379   | 9,219  | 16,595  |
| 0.8                                                | 456 | 3,775 | 10,565  | 10,410 | 18,507  |
| 0.9                                                | 492 | 4,013 | 11,459  | 11,252 | 20,098  |
| 1.0                                                | 523 | 4,286 | 12,641  | 11,619 | 21,335  |

# Results – Generalization



Standard deviation of noise added to demands.



Standard deviation of noise added to diameters.

# Strengths

- Physics-Informed learning:
  - ensures adherence to hydraulic principles,
  - greatly reduces the need for huge datasets,
  - avoids over-fitting to correlations in the dataset,
  - allows out-of-distribution generalization.
- Iterative approach requires only a few GNN layers resulting in a faster lighter model (model parameters < 1M).
- End-to-end differentiable.



# Limitations

- Currently supports:
  - reservoir and consumer junctions,
  - pipes, PRVs and head pumps.
- Can be incorporated:
  - tanks,
  - power pumps and other kinds of valves,
  - other WDS components.





# Challenges in Physics-Informed Learning

# Challenges – Data Normalization

- Different quantities assume different ranges and magnitudes of values.

| WDS          | Anytown | Hanoi  | Fossolo     | Pescara   | Balerna |
|--------------|---------|--------|-------------|-----------|---------|
| No. of nodes | 22      | 32     | 37          | 71        | 447     |
| No. of links | 41      | 34     | 58          | 99        | 454     |
| Heads        |         |        |             |           |         |
| Min          | 3.04    | 86.19  | 107.30      | 50.16     | 97.98   |
| Max          | 81.35   | 100.00 | 121.00      | 57.00     | 127.00  |
| Mean         | 91.82   | 88.48  | 115.96      | 52.76     | 113.30  |
| Demands      |         |        |             |           |         |
| Min          | 2.5e-2  | 4.6e-3 | 2.6e-4      | 3.0e-6    | 0e-0    |
| Max          | 1.2e-1  | 1.0e-1 | 2.0e-3      | 1.0e-2    | 1.5e-3  |
| Mean         | 5.0e-2  | 4.9e-2 | 9.4e-4      | 2.3e-3    | 1.0e-3  |
| Flows        |         |        |             |           |         |
| Min          | 2.5e-3  | 5.8e-3 | 1.7e-6      | 9.9e-13   | 1e-4    |
| Max          | 2.9e-1  | 1.5e-0 | 3.3e-2      | 6.0e-2    | 2.2e-1  |
| Mean         | 4.2e-2  | 2.6e-1 | 3.0e-3      | 7.9e-3    | 1.0e-2  |
| $r$          |         |        |             |           |         |
| Min          | 14.9    | 0.1    | 1.3         | 9.4       | 9.6     |
| Max          | 12932.9 | 652.8  | 208599312.0 | 222,571.3 | 59780.2 |
| Mean         | 1347.6  | 98.5   | 35579844.0  | 33,766.6  | 5840.8  |

# Challenges – Data Normalization

- Data normalization becomes a challenge as physics need to be preserved.

$$\mathbf{d}^* := -\mathbf{A}\mathbf{q}^*, \quad \Delta\mathbf{h}^* := \mathbf{R}^*(\mathbf{q}^*)^x + \mathbf{S}\omega^{*2}\boldsymbol{\iota}^* - \mathbf{S}\boldsymbol{\kappa}^*(\mathbf{q}^*/\omega^*)^{\nu^*}$$

- However, we can show that any normalization of the states (demands, flows, heads, resistance coefficients, etc.) that uses only scalars per state, will preserve the physics per state and will be reversible. For instance:

$$\mathbf{d}^*/\alpha_d := -\mathbf{A}\mathbf{q}^*/\alpha_d, \quad \frac{\Delta\mathbf{h}^*}{\alpha_h\alpha_r\alpha_d^x} := \frac{\mathbf{R}^*}{\alpha_h\alpha_r} \left(\frac{\mathbf{q}^*}{\alpha_d}\right)^x + \mathbf{S}\omega^{*2} \left(\frac{\boldsymbol{\iota}^*}{\alpha_h\alpha_r\alpha_d^x}\right) - \frac{\mathbf{S}}{\omega^{*\nu^*}} \left(\frac{\boldsymbol{\kappa}^*}{\alpha_h\alpha_r\alpha_d^{x-\nu^*}}\right) \left(\frac{\mathbf{q}^*}{\alpha_d}\right)^{\nu^*}$$

- If we chose

$$\begin{aligned} \mathbf{d}_{norm} &= \mathbf{d}^*/\alpha_d, & \mathbf{q}_{norm} &= \mathbf{q}^*/\alpha_d, & \Delta\mathbf{h}_{norm} &= \Delta\mathbf{h}^*/(\alpha_h\alpha_r\alpha_d^x), \\ \mathbf{R}_{norm} &= \mathbf{R}^*/(\alpha_h\alpha_r), & \boldsymbol{\iota}_{norm} &= \boldsymbol{\iota}^*/(\alpha_h\alpha_r\alpha_d^x), & \boldsymbol{\kappa}_{norm} &= \boldsymbol{\kappa}^*/(\alpha_h\alpha_r\alpha_d^{x-\nu^*}), \end{aligned}$$

- The resulting state preserves the physics:

$$\mathbf{d}_{norm} := -\mathbf{A}\mathbf{q}_{norm},$$

$$\Delta\mathbf{h}_{norm} := \mathbf{R}_{norm}\mathbf{q}_{norm}^x + \mathbf{S}\omega^{*2}\boldsymbol{\iota}_{norm} - \mathbf{S}\boldsymbol{\kappa}_{norm}(\mathbf{q}_{norm}/\omega)^{\nu^*}$$

# Challenges – Exploding Gradients

- Physical equation can embody highly non-linear relationships.

- For instance:

$$h_v^* - h_u^* = r_{e_{vu}} \operatorname{sgn}(q_{e_{vu}}^*) |q_{e_{vu}}^*|^{1.852}$$

$$r_{e_{vu}} = 10.667 l_{e_{vu}} \psi_{e_{vu}}^{-4.871} c_{e_{vu}}^{-1.852}, x = 1.852$$

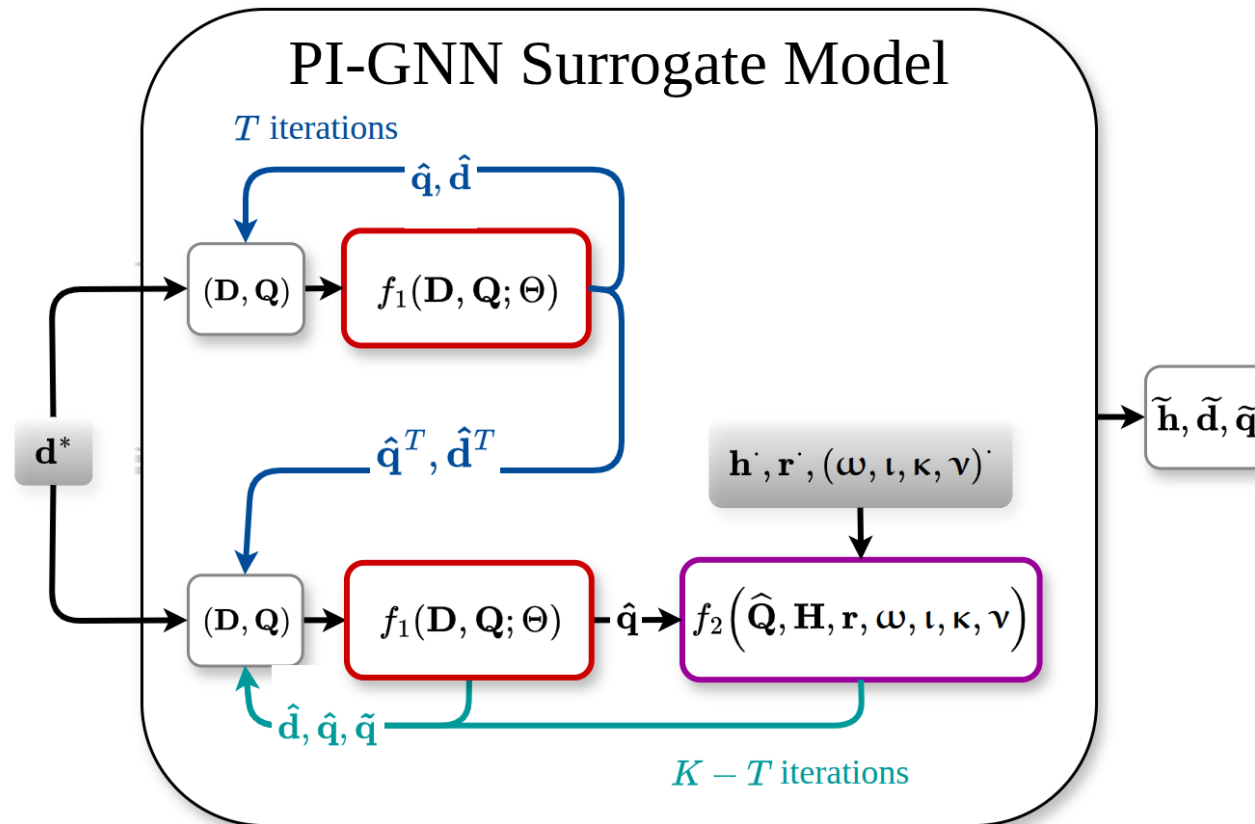
- This can result in highly sensitive gradients during backpropagation.
- One solution is gradient clipping. Add formula



# Towards Digital Twins

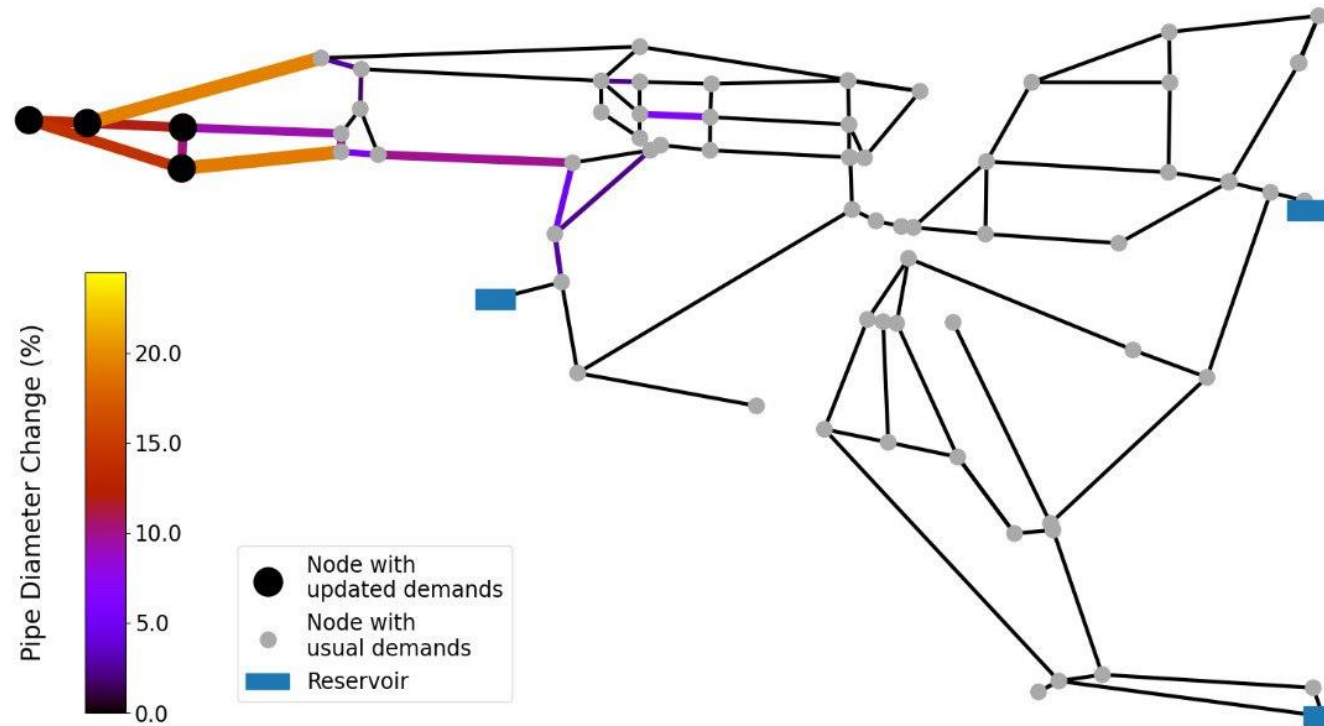
# Differentiability

Any of the inputs can be parametrized and optimized using backpropagation<sup>1</sup>.



# Pipe Diameter Optimization

An increased demand at the black nodes is used to estimate the changes in diameters.







# Hands -On Session



Course material @GitHub  
<https://tinyurl.com/2m4vy8vp>